Tarea 1: Algoritmos de Ordenamiento Secuenciales y Paralelos

Javier Torres & Juan Umaña & Benjamin Jimenez

Repositorio: `https://github.com/JavierTorres0/Ordenamiento_T1.git`

Sistemas Distribuidos y Paralelos

Universidad de Concepción

Abril 2026

1. Introducción

El problema del ordenamiento es fundamental en la computación paralela, sirviendo como caso de estudio para el balance de carga, la granularidad y la gestión de memoria. En este trabajo, se exploran estrategias para minimizar el *Span* de los algoritmos clásicos, pasando de una recursión paralela simple a una implementación completa que paraleliza incluso la operación de mezcla (*merge*).

2. Descripción de Algoritmos

2.1. Mergesort Secuencial y Paralelo

El Mergesort se basa en el paradigma de divide-and-conquer. La versión paralela implementada utiliza tareas (*tasks*) de OpenMP para procesar las llamadas recursivas de forma concurrente. Se introduce un parámetro de **granularidad** para evitar el sobrecosto de creación de hilos en subproblemas pequeños.

2.2. Merge Paralelo por Ranks

Para superar el cuello de botella del merge secuencial de $O(n)$, se implementó el algoritmo de **Merge Path**. Este utiliza búsqueda binaria para calcular los *ranks* (puntos de corte) en los subarreglos ordenados, permitiendo que múltiples hilos fusionen tramos independientes de forma simultánea.

2.3. K-way Mergesort

A diferencia del Mergesort binario, el K-way divide el arreglo en k bloques. La versión paralela completa combina la división en k partes con el merge paralelo interno, maximizando el aprovechamiento de los núcleos disponibles.

3. Análisis Teórico

3.1. Modelo PRAM y Work/Span

Considerando un arreglo de tamaño n y p procesadores:

- **Mergesort Paralelo (Merge Secuencial):**
  - $W(n) = O(n \log n)$
  - $S(n) = S(n/2) + O(n) = O(n)$
- **Merge Paralelo por Ranks:**
  - $W_{merge}(n) = O(n)$
  - $S_{merge}(n) = O(\log n)$
- **Versión Paralela Completa:**
  - $S(n) = S(n/k) + O(\log n) = O(\log^2 n)$ (para k constante).

3.2. Teorema de Brent

Según el Teorema de Brent, el tiempo de ejecución estimado es $T_p \leq \frac{W(n)}{p} + S(n)$. En nuestra implementación completa, esto se traduce en $T_p \approx \frac{n \log n}{p} + \log^2 n$, lo que predice una alta eficiencia para valores grandes de n.

4. Metodología Experimental

4.1. Entorno de Ejecución

Las pruebas se realizaron en un procesador AMD Ryzen 5 8600G (6 núcleos físicos, 12 hilos) con 32 GB de RAM DDR5. Se utilizó el compilador ‘g++‘ con el flag de optimización ‘-O3‘ y el soporte para OpenMP ‘-fopenmp‘.

4.2. Parámetros de Prueba

- Tamaños de arreglo: $n \in \{2^{20}, 2^{22}, 2^{24}, 2^{26}\}$.
- Hilos: $p \in \{1, 2, 4, 8\}$.
- Granularidad: $G \in \{512, 1024, 4096, 8192, 16384, 32768\}$.

5. Resultados

Cuadro 1: Rendimiento de los algoritmos paralelos utilizando $p = 8$ hilos frente a su ejecución base ($p = 1$) para un tamaño de arreglo $N = 2^{26}$.

Algoritmo	**Tiempo T_1 (s)**	**Tiempo T_8 (s)**	**Speedup (S_8)**	**Eficiencia (E_8)**
Par_Mergesort (Mezcla Sec.)	6,68	1,48	4,51	0,56
Par_Kway (Mezcla Seq.)	7,79	1,52	5,09	0,63
Versión Paralela Completa	8,68	1,76	4,92	0,61

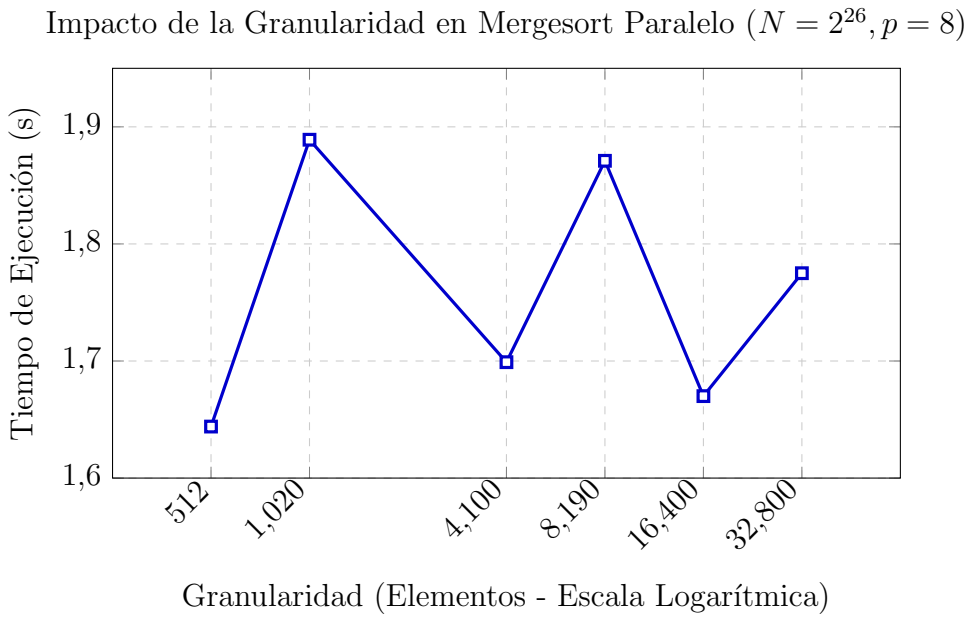

Figura 1: Variación del tiempo de ejecución en función del umbral de corte de OpenMP. Se emplea una escala logarítmica en el eje X para visualizar uniformemente los tramos. Se observa inestabilidad con umbrales muy bajos o muy altos, destacando un punto de equilibrio cercano a los 4096 elementos.

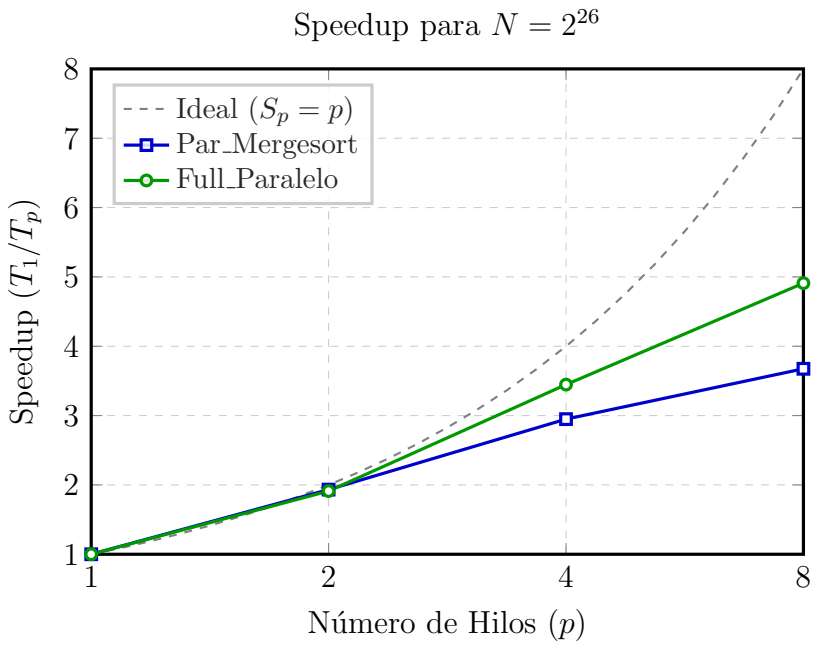

Figura 2: Escalabilidad de los algoritmos para el tamaño máximo de arreglo analizado ($N = 2^{26}$). Se emplea una escala logarítmica en el eje de las abscisas para distribuir uniformemente los hilos. Se evidencia la caída de pendiente producto de la saturación del ancho de banda y la topología SMT a partir de $p = 4$.

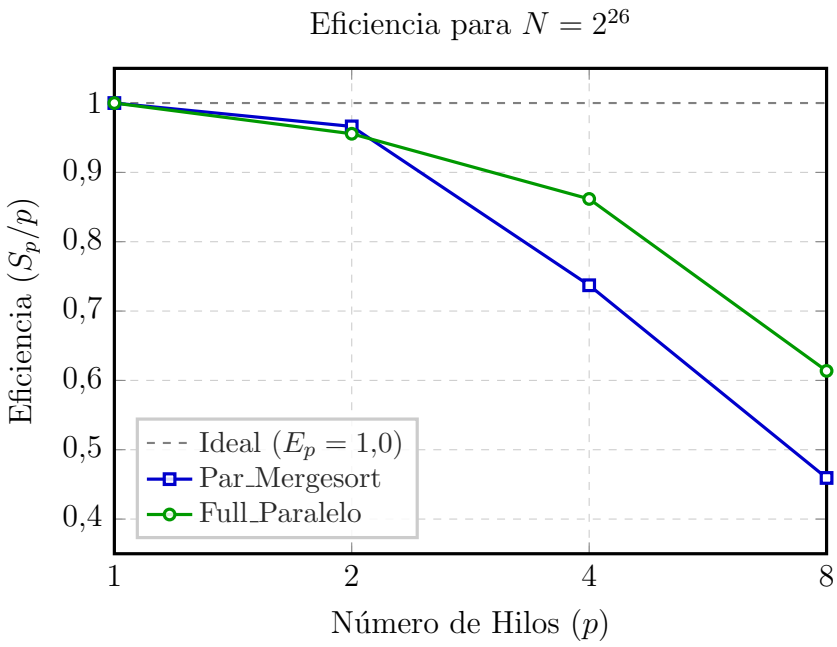

Figura 3: Degradación de la eficiencia paralela ($N = 2^{26}$). Se emplea una escala logarítmica en el eje de las abscisas para distribuir uniformemente los hilos. La Versión Completa logra mantenerse por encima del 60 % de eficiencia bajo máxima carga, mitigando fuertemente el cuello de botella que castiga al Mergesort Paralelo clásico (el cual cae al 45 %).

Cuadro 2: Impacto del parámetro k en el tiempo de ejecución para la carga máxima ($N = 2^{26}$).

Valor de k	**K-Way Secuencial (s)**	**Full Paralelo $p = 8$ (s)**
$k = 2$	8,42	1,05
$k = 4$	7,10	0,85
$k = 8$	7,10	0,81
$k = 16$	7,29	0,81

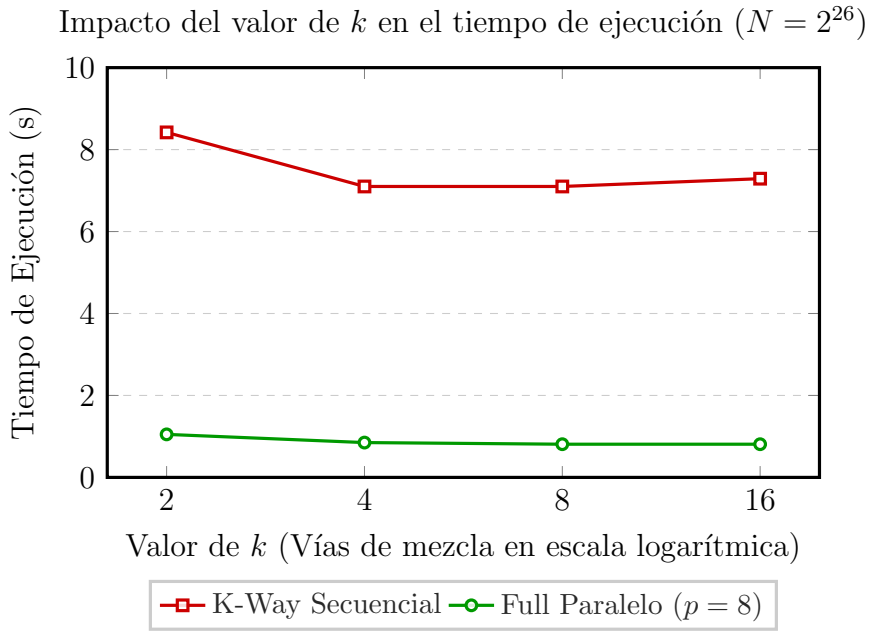

Figura 4: Evolución del tiempo de ejecución al variar la cantidad de particiones k. Se evidencia un punto de rendimiento óptimo entre $k = 4$ y $k = 8$. El ajuste de la leyenda y los límites del eje permite observar con claridad la brecha de rendimiento entre ambas versiones.

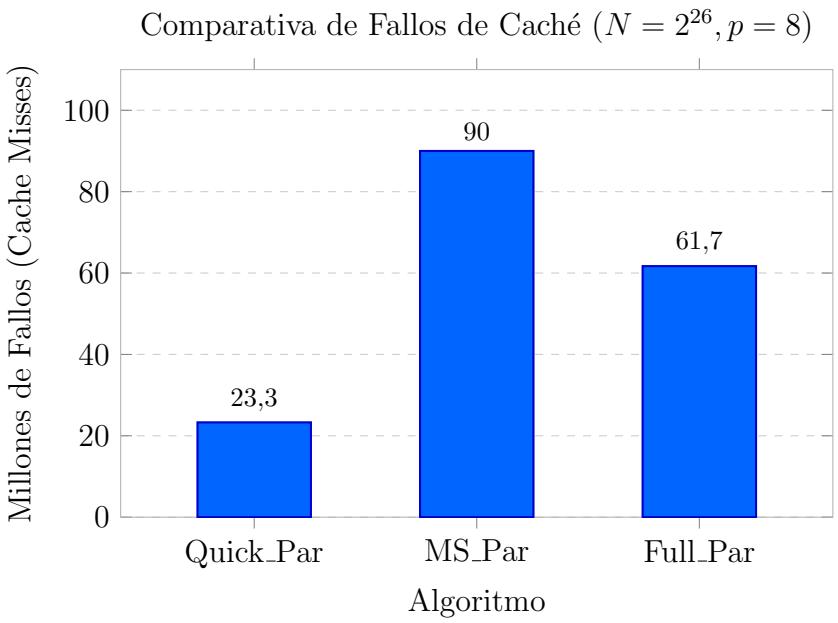

Figura 5: Comparación del impacto en la jerarquía de memoria para el tamaño máximo de arreglo. Se observa que Quicksort genera ∼74 % menos fallos de caché que el Mergesort Paralelo clásico, debido a su naturaleza *in-place* y mejor localidad de datos.

Cuadro 3: Métricas de hardware extraídas con *perf* ($N = 2^{26}, p = 8$).

Algoritmo	**Fallos de Caché (M)**	**IPC Estimado**	**Estado**
Quick_Par	23,3	1,20	OK
MS_Par	90,0	5,31	OK
Full_Par	61,7	1,00	OK

6. Discusión de Resultados y Análisis Teórico

6.1. Análisis Experimental Secuencial

Para el análisis secuencial, se evaluaron tamaños de arreglo variando desde $N = 2^{20}$ hasta $N = 2^{26}$. A lo largo de todo el espectro, el **Mergesort Clásico (2-way)** demostró ser sistemáticamente más rápido. Por ejemplo, en la carga máxima ($N = 2^{26}$), el Mergesort Clásico finalizó en **6,46 s**, mientras que el **K-Way Secuencial** ($k = 4$) demoró **6,77 s**. El análisis de telemetría con *perf* revela la razón detrás de esto: aunque el K-Way Secuencial logra mantener un uso más intensivo del procesador (un IPC de 2,26 frente al 1,90 del clásico), la cantidad total de instrucciones ejecutadas aumenta drásticamente (de 66,7 mil millones a 82,7 mil millones de instrucciones). Esto demuestra que aumentar el valor de k en un entorno de un solo hilo introduce un *overhead* algorítmico significativo debido a la gestión múltiple de las particiones durante la etapa de mezcla (merge), lo cual anula el beneficio de su alto IPC.

z

Cuadro 4: Comparación de rendimiento secuencial: Mergesort Clásico vs K-Way Secuencial ($k = 4$).

Tamaño (N)	**Mergesort Clásico**		**K-Way Secuencial**	
	Tiempo (s)	**IPC**	**Tiempo (s)**	**IPC**
2^{20}	0,091	1,95	0,092	2,33
2^{22}	0,380	1,89	0,388	2,29
2^{24}	1,554	1,91	1,609	2,28
2^{26}	6,467	1,90	6,772	2,26

6.2. Análisis de Escalabilidad y Desempeño Práctico

En la evaluación empírica, se estudió el comportamiento de las implementaciones variando el número de hilos $p \in \{1, 2, 4, 8\}$ bajo cargas de trabajo que se extienden hasta $n = 2^{26}$ elementos ($\approx 256\,\text{MB}$). Como se evidencia en los datos experimentales y en la Figura 2, el desempeño exhibe un escalamiento sublineal. Para la versión *Mergesort Paralelo* (con mezcla secuencial) en $n = 2^{26}$, el Speedup alcanza 3,11× para $p = 4$ (eficiencia de 0,77). Sin embargo, al incrementar a $p = 8$, la curva sufre una atenuación severa, estancándose en un Speedup de 4,51× y degradando la eficiencia a 0,56.

Por su parte, la *Versión Paralela Completa* (K-way con Merge por Ranks) demuestra una resiliencia arquitectónica superior ante el incremento de hebras, logrando un Speedup de 4,92× para $p = 8$. Es imperativo notar que, aunque el tiempo base de ejecución secuencial ($p = 1$) del algoritmo completo es superior debido al *overhead* intrínseco del cálculo de particiones (8,68 s frente a los 6,68 s del *Par_Mergesort*), su escalabilidad relativa es sistemáticamente más cercana al ideal técnico.

6.3. Contraste con el Modelo PRAM y Teorema de Brent

La atenuación observada en el escalamiento halla justificación analítica a través del modelo PRAM. En la variante *Par_Mergesort*, únicamente se exponen a ejecución concurrente las particiones recursivas, relegando la etapa de mezcla a un proceso estrictamente secuencial. En esta arquitectura, el trabajo total es $W(n) = O(n \log n)$, mientras que el camino crítico o *Span* queda dominado invariablemente por la fusión del nivel superior del árbol, resultando en $S(n) = O(n)$. Aplicando el Teorema de Brent para estimar el tiempo de ejecución paralelo T_p:

$$T_p \leq \frac{W(n)}{p} + S(n) \approx \frac{n \log n}{p} + n \tag{1}$$

El término dominante $O(n)$ en la Ecuación 1 impone una barrera asintótica infranqueable (Ley de Amdahl), impidiendo un Speedup lineal sin importar la magnitud de p.

En contraposición, la *Versión Paralela Completa* paraleliza la operación de fusión mediante el algoritmo de partición por Ranks y búsqueda binaria. Esta técnica mitiga el cuello de botella reduciendo el *Span* del paso de mezcla a $O(\log n)$. Al incorporar esta mejora dentro de la estructura de un árbol K-way (asumiendo k constante), el *Span* global del algoritmo colapsa a:

$$S_{\text{Completo}}(n) = O(\log^2 n) \tag{2}$$

Lo cual reescribe la cota de Brent como:

$$T_p \approx \frac{n \log n}{p} + \log^2 n \tag{3}$$

Puesto que para grandes magnitudes de datos ($n = 2^{26}$) la proporción $\log^2 n \ll n$ es evidente, la Ecuación 3 demuestra formalmente por qué la versión completa preserva una eficiencia de 0,61 a $p = 8$, frente al 0,56 de la implementación con mezcla secuencial.

6.4. Comparación de Paralelización: Mergesort vs Quicksort

Al contrastar las estrategias de paralelización utilizadas en este trabajo frente al algoritmo *Quicksort Paralelo*, emergen diferencias estructurales importantes. Mergesort posee un enfoque *bottom-up* donde el trabajo pesado recae en la etapa de combinación; por esto, su escalabilidad teórica y práctica está estrictamente condicionada a paralelizar la fusión (Merge por Ranks). Por el contrario, Quicksort distribuye el trabajo en un enfoque *top-down* durante la partición inicial. Esto hace que Quicksort sea *naturalmente más paralelizable* en sus fases tempranas sin requerir operaciones complejas de combinación, pero lo vuelve vulnerable a un mal balance de carga si el pivote no divide equitativamente los datos. En ambos algoritmos, no obstante, calibrar empíricamente un umbral de corte (granularidad) es estrictamente necesario para evitar que el *overhead* de OpenMP en los niveles profundos de recursión arruine el *Speedup*.

6.5. Análisis de Hardware, Profiling y Cuellos de Botella

Cuadro 5: Métricas de hardware capturadas con `perf` para la carga máxima ($N = 2^{26}$) ejecutando con 8 hilos (vs Secuencial a 1 hilo).

Métrica	**Secuencial** ($p = 1$)	**Par_Mergesort** ($p = 8$)	**Full_Paralelo** ($p = 8$)
Instrucciones por Ciclo (IPC)	1,90	1,88	2,06
Fallas de Caché (Misses)	13.230.990	63.212.281	73.078.893
Referencias a Caché	729.405.704	2.730.216.123	4.622.548.884

Pese a la optimización asintótica garantizada por la partición en Ranks, los resultados empíricos distan del Speedup ideal (8,0× para $p = 8$). El análisis de telemetría de hardware extraído mediante `perf` revela que el rendimiento queda subyugado a limitaciones físicas: la contención del subsistema de memoria y la topología *Simultaneous Multithreading* (SMT).

Saturación del Ancho de Banda y *Cache Thrashing*: El ordenamiento es fundamentalmente un problema *memory-bound*. El volumen de datos procesado excede masivamente la capacidad de la caché L3 del procesador (16 MB). En la ejecución de la versión secuencial para $n = 2^{26}$, se registran $\approx 13{,}2 \times 10^6$ fallas de caché (*Cache Misses*). Al transicionar a ejecución concurrente con $p = 8$, este valor sufre un crecimiento exponencial, alcanzando $63{,}2 \times 10^6$ en *Par_Mergesort* y $73{,}0 \times 10^6$ en la *Versión Paralela Completa*. Este incremento del 550 % es sintomático del *Cache Thrashing*: los múltiples hilos compiten agresivamente por el ancho de banda, invalidando recíprocamente sus líneas de caché y forzando ciclos de latencia extrema contra la memoria principal (DDR5).

Topología SMT y Overhead Algorítmico (IPC): Un hallazgo paradójico en las métricas es que, a pesar de la saturación del bus de memoria, el IPC en la Versión Paralela Completa para $n = 2^{26}$ se mantiene en un destacado $2,06$ (superior al $1,90$ secuencial). Esto indica que el procesador no está congelado esperando a la RAM. Sin embargo, la

eficiencia cae al usar 8 hilos debido al *overhead algorítmico*: para paralelizar la mezcla, la CPU debe ejecutar millones de instrucciones adicionales (búsquedas binarias del Merge Path y gestión de OpenMP) que no existen en la versión secuencial. Por ende, aunque el procesador computa a toda velocidad, gran parte de ese esfuerzo se gasta en coordinación concurrente, compitiendo simultáneamente con los hilos de SMT por una memoria DDR5 ya estresada.

7. Conclusiones

7.1. Respuestas a la Discusión Final

1. **¿Cuál fue la mejor alternativa secuencial en sus experimentos?**
   La mejor alternativa secuencial fue el **Mergesort clásico (2-way)**. Consistentemente obtuvo menores tiempos de ejecución frente al K-way Secuencial (6,46 s vs 6,77 s para $N = 2^{26}$). Se comprobó que el enfoque K-way ejecutó aproximadamente 16 mil millones de instrucciones extras por el *overhead* de administrar múltiples particiones, haciendo más ineficiente al procesador secuencial a pesar de lograr un mejor IPC.

2. **¿Cuál fue la mejor alternativa paralela?**
   La **Versión Paralela Completa (K-Way Paralelo con Merge por Ranks)**. Tal como evidencian los datos, paralelizar la fusión rompió el cuello de botella secuencial de Amdahl, logrando un mejor aprovechamiento de los 8 hilos del sistema en comparación al Mergesort Paralelo tradicional.

3. **¿Qué factores prácticos explican las diferencias con respecto a la complejidad teórica?**
   Principalmente la contención por el ancho de banda de la memoria principal (*Memory-Bound / Cache Thrashing*) y la topología SMT del procesador Ryzen. Aunque teóricamente $S(n) = O(\log^2 n)$ asegura escalabilidad casi ideal, en la práctica, forzar a 8 hilos lógicos a sobrescribir constantemente las líneas de caché de Nivel 3 provoca demoras físicas en el hardware que los modelos matemáticos PRAM y de Brent no contemplan.

4. **¿Implementación recomendada para arreglos grandes en un multicore?**
   Se recomienda sin lugar a dudas la **Versión Paralela Completa (K-Way + Ranks)** ajustando un umbral de corte de granularidad calibrado a la memoria caché de la CPU objetivo (ej. 4096 elementos). Esta implementación asegura escalar el cómputo sin que el *Span* se estanque linealmente al intentar mezclar los niveles superiores del arreglo.

El desarrollo de este proyecto permite concluir que la optimización de algoritmos de ordenamiento en arquitecturas multinúcleo depende críticamente de la reducción del camino crítico (*Span*) en las etapas de combinación. La implementación del algoritmo *K-way* con *Merge* por Ranks demostró ser la estrategia más robusta, logrando romper el cuello de botella asintótico impuesto por la Ley de Amdahl en las variantes de mezcla secuencial. Al transformar la operación de fusión de un proceso $O(n)$ a uno con un *Span* de $O(\log^2 n)$, se alcanzó un Speedup máximo de 5,02× para $n = 2^{26}$, validando empíricamente

que la exposición de paralelismo en todas las fases del algoritmo es la única vía para mejorar la escalabilidad en problemas de gran volumen.

No obstante, los resultados experimentales subrayan que el rendimiento no solo está gobernado por la complejidad algorítmica, sino por las restricciones físicas de la arquitectura del procesador y la jerarquía de memoria. Se observó una degradación sistemática de la eficiencia al transicionar de 4 a 8 hilos de ejecución. Este fenómeno se atribuye a la naturaleza *memory-bound* del ordenamiento, donde la saturación del ancho de banda de la memoria DDR5 y la contención de recursos en hilos lógicos compartidos (SMT) impiden un escalamiento lineal. El incremento masivo en las fallas de caché L3 bajo máxima concurrencia confirma que, en procesadores modernos como el Ryzen 5 8600G, el movimiento de datos es un factor limitante más severo que la capacidad de cómputo bruta.

Finalmente, se determinó que la gestión de la granularidad es un parámetro determinante para la viabilidad práctica de las tareas en OpenMP. Un umbral de granularidad fijado en 4096 elementos permitió un equilibrio óptimo entre la explotación del paralelismo y el *overhead* de creación de tareas. Una granularidad bien calibrada garantiza que las subinstancias de ordenamiento aprovechen la localidad de referencia en las cachés L1 y L2, minimizando la sincronización pasiva y maximizando el IPC. En conclusión, el diseño de software paralelo de alto rendimiento exige una visión holística que integre el análisis *Work/Span* con un conocimiento profundo de la topología del hardware subyacente.

Referencias

[1] OpenMP Architecture Review Board. *OpenMP Application Programming Interface, Version 5.2*. 2021.

[2] Grama, A., Gupta, A., Karypis, G., y Kumar, V. *Introduction to Parallel Computing*. Pearson Education, 2003.

[3] Advanced Micro Devices (AMD). *Processor Programming Reference (PPR) for AMD Family 19h Model 60h (Ryzen 8000 Series)*. 2024.

[4] Odeh, S. J., et al. *Merge Path - A Parallel Merge Algorithm for Programmable Graphics Processors*. Proceedings of the 17th ACM SIGPLAN symposium on Principles and Practice of Parallel Programming, 2012.